A Cyclic Calculus of Inductive Constructions: Escaping the Bureaucracy of Inductive/Recursive Syntax

(Scidonia/cyclic notes)

February 9, 2026

Abstract

This note motivates a cyclic variant of a Calculus of Inductive Constructions (CIC)-like calculus in which proofs are finite graphs with cycles, validated by a global progress condition, rather than trees generated by a fixed family of induction principles. This representation supports normalization-style transformations (commuting conversions, unfolding/refolding, etc.) that can *identify* proofs that are distinct syntactically, yielding a form of post-hoc proof identity. We outline (i) the source term calculus, (ii) a fix-free cyclic proof/term language with explicit substitution evidence, (iii) the global condition on cycles, and (iv) the use of *supercompilation* as a normalization procedure.

Contents

1 Introduction

The design goal of this project is not simply to add "cycles" for convenience. The goal is to change what we treat as *primitive* in a proof calculus.

In conventional systems, recursive reasoning is packaged as a fixed-point operator or as a family of induction/recursion schemata. Those choices are powerful, but they also hard-code a particular presentation of recursion into the syntax of proofs. As a result, proofs that are semantically equivalent can remain syntactically far apart, even after many semantics-preserving transformations.

Cyclic proof theory suggests a different decomposition:

> A proof is a finite graph of locally-correct steps; recursion is represented by cycles; and soundness is ensured by a global progress condition.

This decomposition has an attractive consequence for proof engineering and normalization: we can refactor recursive structure (fold/refold), commute conversions, and perform administrative normalization without being forced to preserve the exact boundaries of `fix` binders or induction schemata. In the best case, normalization can reveal that two proofs are not just equivalent, but *the same cyclic object* after refolding.

The rest of the paper outlines:

- the source CIC-like term calculus (Section 4);
- the cyclic proof/term graph architecture (Section 5);
- the global condition on cycles (Section 6);
- and representative transformations like CaseCase and head beta-reduction (Section 7).

2 Prior work and relation to Cyclic CIC

This section positions the present development relative to (i) cyclic proof theory, (ii) proof-theoretic treatments of induction via global discharge conditions, and (iii) CIC kernel engineering.

Cyclic proof theory: from concrete systems to abstract frameworks. Cyclic proof systems represent potentially infinite derivations using finite graphs with back-edges, validated by a global soundness condition. This approach originates in cyclic proofs for inductive definitions and infinite descent [3, 4]. Afshari and Wehr develop a modern, highly abstract account of cyclic proof checking based on trace conditions, activation algebras, and categorical structure [1]. Our work is complementary in two ways: (1) we instantiate cyclic proofs inside a typed setting (a CIC-like calculus with evaluation and CIU (*closed instantiations of uses*) observational equivalence); (2) we focus on using cyclic proofs as *normal forms stable under program/proof transformations* (commuting conversions, unfold/refold, etc.). In particular, many of our proofs of interest are about preservation of observational equivalence under transformations rather than about the abstract complexity of proof checking.

Local induction vs global discharge. Sprenger and Dam compare a proof system with a local well-founded induction rule to a cyclic system validated by a global induction discharge condition, and give effective translations between the two (in the setting of the first-order μ-calculus) [8]. We use this as direct prior evidence for the core design thesis of this project: *the induction principle need not be fixed syntactically.* Where Sprenger–Dam study the relationship between local and global presentations of induction in a fixed-point logic, we pursue the same decomposition in a dependent type-theoretic setting, and then exploit it operationally: cycles become the unit of fold/refold during normalization and compilation of proof objects.

Engineering CIC kernels. Asperti et al. give a detailed kernel-level account of CIC checking in the Matita prover, emphasizing compactness and clear engineering boundaries [2]. Our contribution is not a new kernel design; instead, we propose a proof-object language where recursive structure is externalized as graph structure plus a global condition. Nevertheless, the kernel perspective is relevant: if cyclic proofs are to support a strong notion of post-transformation proof identity, then proof checking and definitional equality need representations that survive aggressive normalization and refolding.

Supercompilation and termination control. Supercompilation is a semantics-driven program transformation methodology centered on symbolic evaluation ("driving"), generalization, and fold/refold steps [9, 7]. A key practical ingredient is a *control* (or "whistle") mechanism preventing infinite unfolding, often based on homeomorphic embedding and Kruskal-style well-quasi-ordering arguments [5, 6]. Our stance is that cyclic proof normalization is *precisely supercompilation*—but performed at the level of typed judgements/proof objects rather than untyped programs. The global cycle condition and the control relation play the same role they do in supercompilation: they justify refolding and prevent uncontrolled infinite unfolding.

3 Why go cyclic? Proof identity and non-fixed induction principles

We begin with tiny examples that make the motivation concrete: in ordinary CIC developments, the objects we want to treat as "the same proof" or "the same program" are often separated by

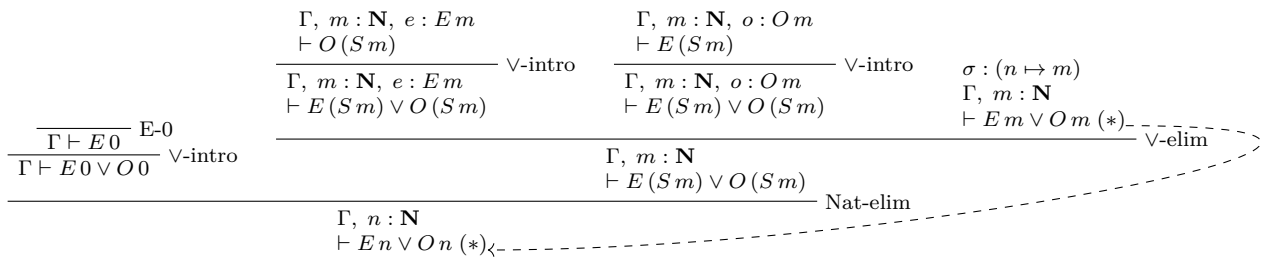

Figure 1: A schematic cyclic natural-deduction proof in CIC style (example shape due to [3]). Write $E\,n$ and $O\,n$ for evenness/oddness judgements over $\mathbf{N}$. We treat $A \vee B$ as an inductive type; proof terms are elided. The bud $(*)$ is discharged by a back-link to its companion, carrying explicit substitution evidence $\sigma : (n \mapsto m)$.

```
Definition pipeline_vec_raw (l : list A)
  : Vector.t C (length (map g (map f l))) :=
  Vector.of_list (map g (map f l)).

Definition pipeline_vec (l : list A) : Vector.t C (length l) :=
  eq_rect _ (fun n => Vector.t C n) (pipeline_vec_raw l)
          _ (length_map_map l).
```

Figure 2: Index bureaucracy caused by list pipeline composition: the index $\mathsf{length}(\mathsf{map}\ g(\mathsf{map}\ f\ l))$ must be transported to $\mathsf{length}\ l$ using an explicit equality proof.

administrative syntax. Cyclic proof objects aim to eliminate this bureaucracy by making recursion a matter of *graph structure* (cycles + explicit substitutions), leaving local typing and computation unchanged.

3.1 Example 1: cyclic proofs as proof graphs

Figure 1 shows the shape of a cyclic proof as a finite natural-deduction derivation with a back-link from a bud to its companion, carrying explicit substitution evidence.

Definition 3.1 (Cyclic proof (after [3], informal))**.** *A* cyclic preproof *is a finite proof graph whose nodes are labelled by judgements and locally justified by inference rule instances; some leaves (*buds*) are discharged by* back-links *to earlier occurrences of the same obligation (their* companions*). In our setting, each back-link additionally carries explicit substitution evidence describing how the bud is instantiated from its companion.*

A cyclic proof *is a cyclic preproof equipped with a* global trace/progress condition *ensuring that the infinite unfolding obtained by repeatedly following back-links is sound (Section 6).*

3.2 Example 2: dependent index bureaucracy

In dependently typed developments, index expressions are themselves programs. Even when these programs are total and terminating, definitional equality may not normalise them using fusion/algebraic laws, forcing explicit transports/casts. A representative pattern is:

These examples motivate the rest of the paper: cyclic proofs aim to support semantics-preserving transformations (supercompilation-style driving plus fold/refold under a termination control) that

eliminate administrative structure and yield canonical cyclic representatives.

A standard way to represent recursive reasoning in type theory is to bake recursion into the term language:

- either directly, via a fixed-point constructor (`fix`), or
- indirectly, via a pre-chosen set of induction/recursion schemata tied to the type formers.

This choice has a subtle but important consequence: it commits the system to a *particular syntactic presentation* of recursive reasoning. Two proofs may establish the same semantic property, but remain permanently different pieces of syntax because they encode recursion using different unfoldings, different induction schemata, or different administrative structure.

Of course, there cannot be a single fully automatic principle that always produces a canonical representative for *all* proofs and programs of interest. However, in many useful cases one can normalize aggressively enough (using a supercompilation-style bundle of semantics-preserving transformations, plus a termination control) that distinct presentations converge to the same cyclic normal form (up to a chosen graph equivalence). This is precisely where the approach pays off: it avoids proof bureaucracy in the common situation where the remaining differences are administrative rather than semantic.

3.3 The core motivation: induction principles need not be fixed

Cyclic proof theory takes a different stance:

> The recursive structure of a proof is not a primitive binder or a fixed schema; it is a *cycle* in a finite proof graph.

A cycle does not, by itself, guarantee soundness. Instead we equip the finite graph with a *global progress condition* (Section 6) that rules out "bad" cycles and validates the intended infinite unfolding.

The key conceptual gain is *modularity of recursion*: we can change the cyclic structure (by folding, refolding, or commuting constructors through eliminators) without changing the local steps of the proof. In a fixed-point calculus, the recursion principle is pinned to the syntax of `fix`; in a schema-based setting it is pinned to the chosen eliminators. In a cyclic calculus, the recursion principle is an emergent property of the graph plus the global condition.

3.4 Post-transformation proof identity

This flexibility is not merely aesthetic. It enables a strong normalisation/canonicalisation story for proofs:

- Start from two proofs that are syntactically different (for example: one uses an unfolding step early, another delays it; one distributes a case earlier; one uses a different cycle boundary).
- Apply a sequence of semantics-preserving transformations (Section 7).
- After normalization/refolding, the resulting cyclic proof graphs may become *isomorphic* or even *literally identical* as finite objects.

In other words, transformations can expose that two proofs are "the same proof" once recursion is treated as *structure* (cycles) rather than as a fixed syntactic operator or schema. This is precisely the kind of identification that is difficult in traditional settings: the syntactic boundary of a `fix`

binder (or the syntactic choice of induction schema) tends to be rigid, so normalization can preserve semantics while still leaving behind irreducible syntactic differences.

Cyclic proofs aim to make such equivalences more intensional: if two proofs normalize to the same cyclic representative (up to a chosen notion of graph equivalence such as bisimulation), we can treat them as equal in a strong, structurally meaningful sense.

4 Source calculus (terms, typing, semantics)

This section summarizes the *source* term language and its typing and operational semantics. It corresponds closely to:

- term syntax: `theories/Syntax/Term.v`
- typing: `theories/Judgement/Typing.v` (inductive judgement `has_type`)
- call-by-name semantics: `theories/Semantics/Cbn.v`

4.1 Term syntax

We write terms t, u and types A, B using the following constructors (de Bruijn indices for variables):

$$\begin{aligned} t \quad ::= \quad & \mathsf{Var}\, x \mid \mathsf{Sort}\, i \mid \Pi(A).\, B \mid \lambda(A).\, t \mid t\, u \mid \mathsf{fix}(A).\, t \\ & \mid \mathsf{Ind}^I(\overline{a}) \mid \mathsf{roll}^I_c(\overline{a}) \mid \mathsf{case}^I(t; C; \overline{br}) \end{aligned}$$

Indexed inductive families. The system supports CIC-style indexed inductive families. An inductive $\mathsf{Ind}^I(\overline{a})$ is applied to arguments $\overline{a}$ (parameters and indices). Constructors $\mathsf{roll}^I_c(\overline{a})$ take all arguments (non-recursive and recursive) uniformly as $\overline{a}$. The dependent case eliminator $\mathsf{case}^I(t; C; \overline{br})$ has a *dependent motive* C binding the scrutinee: under the binder, C has access to $x : \mathsf{Ind}^I(\overline{idx})$ where $\overline{idx}$ are the scrutinee's indices.

4.2 Typing

A typing environment Σ stores inductive signatures (strictly-positive definitions). A context Γ is a list of types. The main judgement is $\Sigma;\, \Gamma \vdash t : A$.

Below is the core typing system (informally mirroring `Typing.has_type`). We omit the full inductive signature side-conditions and arity bookkeeping, but include the essential shape.

$$\frac{\Gamma(x) = A}{\Sigma;\, \Gamma \vdash \mathsf{Var}\, x : A}\ (\textsc{TyVar})$$

$$\frac{}{\Sigma;\, \Gamma \vdash \mathsf{Sort}\, i : \mathsf{Sort}\, i + 1}\ (\textsc{TySort})$$

$$\frac{\Sigma;\, \Gamma \vdash A : \mathsf{Sort}\, i \qquad \Sigma;\, A :: \Gamma \vdash B : \mathsf{Sort}\, j}{\Sigma;\, \Gamma \vdash \Pi(A).\, B : \mathsf{Sort}\ \max(i, j)}\ (\textsc{TyPi})$$

$$\frac{\Sigma;\, \Gamma \vdash A : \mathsf{Sort}\, i \qquad \Sigma;\, A :: \Gamma \vdash t : B}{\Sigma;\, \Gamma \vdash \lambda(A).\, t : \Pi(A).\, B}\ (\textsc{TyLam})$$

$$\frac{\Sigma;\, \Gamma \vdash t : \Pi(A).\, B \qquad \Sigma;\, \Gamma \vdash u : A}{\Sigma;\, \Gamma \vdash t\, u : B[u/0]}\ (\textsc{TyApp})$$

$$\frac{\Sigma;\, \Gamma \vdash A : \mathsf{Sort}\, i \qquad \Sigma;\, A :: \Gamma \vdash t : A \uparrow}{\Sigma;\, \Gamma \vdash \mathsf{fix}(A).\, t : A}\ (\textsc{TyFix})$$

Typing indexed inductives. Let $\Sigma(I) = (\overline{P}, \overline{X}, \ell, \overline{ctor})$ where:

- $\overline{P}$ are uniform parameters
- $\overline{X}$ are index types
- ℓ is the universe level
- $\overline{ctor}$ are constructor specifications

Each constructor c has:

- Non-recursive argument types $\overline{\tau}_{param}$
- Recursive argument specifications (indices at which recursion occurs)
- Result indices $\overline{idx}_c$

Typing rules (informal):

- $\mathsf{Ind}^I(\overline{a})$ has type $\mathsf{Sort}\, \ell$ if $\overline{a}$ matches $\overline{P} +\!+ \overline{X}$
- $\mathsf{roll}^I_c(\overline{a})$ has type $\mathsf{Ind}^I((\overline{p} +\!+ \overline{idx}_c[\overline{a}]))$ where $\overline{p}$ are the parameter values
- $\mathsf{case}^I(t; C; \overline{br})$:
  - Scrutinee: $\Sigma;\, \Gamma \vdash t : \mathsf{Ind}^I(\overline{scrutIdx})$
  - Motive: $\Sigma;\, \Gamma, x : \mathsf{Ind}^I(\overline{scrutIdx}) \vdash C : \mathsf{Sort}\, i$ (dependent on scrutinee)
  - Each branch: $\Sigma;\, \Gamma \vdash br_c : \Pi\overline{\tau}_c.\, C[\mathsf{roll}^I_c(\overline{a})/x]$
  - Result type: $C[t/x]$

See `theories/Syntax/StrictPos.v` for the signature representation and `theories/Judgement/Typing.v` for the full typing rules.

4.3 Call-by-name operational semantics

We use a weak-head, call-by-name reduction relation $t \to u$ (see `Semantics/Cbn.v`).

Key rules include:

$$\frac{}{(\lambda(A).\, t)\, u \to t[u/0]}\ (\textsc{Step-Beta})$$

$$\frac{t \to t'}{t\, u \to t'\, u}\ (\textsc{Step-App1})$$

$$\frac{}{\mathsf{fix}(A).\, t \to t[\mathsf{fix}(A).\, t/0]}\ (\textsc{Step-Fix})$$

$$\frac{t \to t'}{\mathsf{case}^I(t; C; \overline{br}) \to \mathsf{case}^I(t'; C; \overline{br})}\ (\textsc{Step-CaseScrut})$$

$$\frac{branch(\overline{br}, c) = br}{\mathsf{case}^I((\mathsf{roll}^I_c(\overline{a})); C; \overline{br}) \to (br\ \overline{a})}\ (\textsc{Step-CaseRoll})$$

In the Step-CaseRoll rule, the runtime step selects the branch and applies it to the constructor arguments. Dependence is tracked by typing: the case expression has type $C[t/x]$ (i.e. the motive instantiated with the scrutinee), even though the motive does not appear at runtime.

We write $t \to^* u$ for the reflexive-transitive closure of $\to$. Values are λ-abstractions and constructor rolls.

5 Cyclic CIC objects (proof graphs and fix-free vertex language)

There are two related layers in the development:

(1) a generic notion of *preproof / cyclic proof* as a finite digraph whose nodes are labelled by judgements;

(2) a concrete *fix-free* vertex language (nodes like `nLam`, `nApp`, `nBack`) used by read-off/extraction.

5.1 Preproofs: locally-correct finite proof graphs

A preproof is a finite directed graph together with:

- a node-label function $\mathsf{label} : V \to \mathsf{Judgement}$
- a successor function $\mathsf{succ} : V \to \mathrm{list}\ V$
- a local correctness property stating that each node is justified by a rule instance: $\mathsf{Rule}(\mathsf{label}(v),\ \mathrm{map}\ \mathsf{label}\ (\mathsf{succ}($

This corresponds to `theories/Preproof/Preproof.v`. A *rooted* preproof additionally designates a root vertex.

5.2 Cyclic proofs: global condition + witness

A cyclic proof packages a preproof together with an additional witness W and a predicate $\mathsf{progress\_ok}$ establishing the global soundness condition. See `theories/CyclicProof/CyclicProof.v`.

5.3 Semantics and intended equality

For the purposes of this paper, we take the meaning of a cyclic object to be given by its *extracted term*. That is, a cyclic proof/term graph p denotes the source-term $\mathsf{extract}(p)$ produced by the read-off/extraction pipeline (Section 8).

Accordingly, the intended equality of cyclic objects is not raw graph isomorphism but *semantic equality of extracted terms*: we regard p and q as representing the same proof/program when their extracted terms are equivalent up to typed CIU:

$$p \approx q \quad \overset{\text{def}}{\iff} \quad \mathsf{ciu\_jTy}(\Sigma, \Gamma, \mathsf{extract}(p), \mathsf{extract}(q), A)$$

for the relevant typing judgement. This choice is deliberate: it lets us talk about canonical representatives in a way that matches the mechanized observational semantics, and it avoids committing to a particular quotient on finite graphs before the refolding theory is finalized.

5.4 Fix-free vertex language (used by read-off)

The read-off compiler produces a fix-free cyclic term graph whose node labels are:

$$\begin{array}{l} \texttt{nVar}\ x \mid \texttt{nSort}\ i \mid \texttt{nPi} \mid \texttt{nLam} \mid \texttt{nApp} \\ \texttt{nInd}\ I \mid \texttt{nRoll}\ (I, c, n_p, n_r) \mid \texttt{nCase}\ (I, n_{br}) \\ \texttt{nSubstNil}\ k \mid \texttt{nSubstCons}\ k \mid \texttt{nBack} \end{array}$$

Key design points (see `theories/Transform/ReadOff.v`):

- There is *no* `fix` node label.
- Recursion is represented by `nBack` nodes, which point to: (i) a cycle-target vertex, and (ii) an explicit substitution-evidence vertex.
- Substitutions are represented as linked lists of evidence nodes: `nSubstNil` and `nSubstCons`, mirroring list-backed substitutions.

5.5 Judgements over vertices

A first-cut judgement language over vertices (see `theories/Transform/CyclicRules.v`) is:

$$\mathsf{jTy}\ \Gamma\ v\ A \quad \mid \quad \mathsf{jEq}\ \Gamma\ v\ w\ A \quad \mid \quad \mathsf{jSub}\ \Delta\ sv\ \Gamma$$

Intuitively:

- $\mathsf{jTy}\ \Gamma\ v\ A$: vertex v has type-vertex A under context of vertices Γ.
- $\mathsf{jSub}\ \Delta\ sv\ \Gamma$: substitution vertex sv is well-typed as a substitution from Γ into Δ.
- jEq provides intensional equality structure (initially refl/symm/trans; computation rules are planned).

The rules reference abstract graph-level operations shiftV and substV, intended to be implemented by fix-free rewriting (or by derived operations on the graph).

Backlink rule (core idea). A backlink node `nBack` represents a recursive call as an instantiation of an earlier obligation:

$$\frac{\mathsf{jTy}\ \Gamma_0\ \mathit{target}\ A_0 \qquad \mathsf{jSub}\ \Gamma\ \mathit{sv}\ \Gamma_0}{\mathsf{jTy}\ \Gamma\ \mathit{back}\ (\mathsf{substV}\ \mathit{sv}\ A_0)}\ (\textsc{TyBack})$$

This is where recursion lives in the cyclic object: the graph edge from *back* to *target* closes the cycle, while *sv* records how the recursive call is instantiated.

6 Global conditions on cycles

Local correctness is not enough: an arbitrary cyclic preproof may be unsound. Cyclic proof theory therefore adds a *global* condition to rule out "bad" cycles.

The repository currently develops a ranking-style template (see `theories/Progress/Ranking.v`).

6.1 Ranking condition (template)

Fix a finite digraph G with vertices V. Assume:

- a predicate $\mathsf{progress\_edge}(v, w)$ selecting edges that must make strict progress,
- a ranking domain M with well-founded strict order $<_M$,
- a rank function $\mathsf{rank} : V \to M$.

The ranking condition requires:

(a) **Well-foundedness**: $<_M$ is well-founded.

(b) **Monotonicity**: along every edge $v \to w$, ranks do not increase.

(c) **Strictness on progress edges**: if $v \to w$ is a progress edge, then $\mathsf{rank}(w) <_M \mathsf{rank}(v)$.

(d) **Every directed cycle has progress**: every directed cycle in G contains at least one progress edge.

6.2 Why this matters for transformations

Transformations that introduce, remove, or restructure cycles (fold/refold, read-off compilation, or higher-level normalisation steps) are only meaningful if the resulting cyclic object still has a progress witness.

Conceptually:

> Finite graph + local rule correctness + global cycle progress ⇒ sound infinite unfolding/meaning.

Thus, semantic preservation results for transformations typically factor into:

(1) a *local* preservation argument (the transformation preserves the intended judgement/evaluation meaning), and

(2) a *global* argument transporting the progress witness (or rebuilding one).

7 Transformations: CaseCase, head beta, unfold/refold

This section outlines the main transformations currently being developed at the term level, with the intended lift to cyclic proof objects via extraction. The motivating theme is not merely that these rewrites preserve observational equivalence, but that they support a notion of *canonical cyclic representatives*.

7.1 Supercompilation (what it consists of)

In this development, "supercompilation" is used in the classical sense of semantics-driven normalization by a bundle of CIU-preserving transformations. Concretely, it consists of the following ingredients, applied iteratively under a termination control (a "whistle"):

1. **Partial evaluation** (driving/head reduction): evaluate known redexes (beta, fix-unfolding, and case-of-constructor) to expose computation.
2. **Information propagation**: push known information (especially from scrutinees) into motives/branches so later transformations can fire.
3. **Case distribution** (case commuting conversions): redistribute/delay case analysis by commuting nested cases (CaseCase).
4. **Unfolding**: expand definitions/cycles to expose new redexes (term-level fix unfolding, and graph-level unfolding of back-links/sharing).
5. **Generalisation**: when unfolding would diverge, replace a configuration by a more general one (guided by embedding/wqo arguments).
6. **Folding/refolding**: detect repeated (generalised) configurations and introduce sharing/back-links again.

The remainder of the section spells out the term-level instances of these steps and how they connect to the cyclic setting.

7.2 Beta-reduction strategies

The file `theories/Transform/BetaReduce.v` defines several reduction strategies:

Single-step head reduction.

$$\mathsf{beta\_reduce\_once}(t) \triangleq \begin{cases} (\lambda(A).\, body)\ u \mapsto body[u/0] & \text{if } t = (\lambda(A).\, body)\ u \\ t & \text{otherwise.} \end{cases}$$

Weak-head normal form (WHNF). $\mathsf{whnf\_reduce}(n, t)$ iterates head reductions until reaching WHNF or exhausting fuel n. This reduces only at the head position, matching call-by-name semantics.

Full normalization. $\mathsf{full\_normalize}(n, t)$ reduces everywhere, including under binders:

- First reduces to WHNF at the head
- Then recursively normalizes all subterms
- Descends under λ, Π, `fix`, and case motives

The development proves a CIU-style correctness theorem for head reduction:

$$\Sigma;\, \Gamma \vdash t : A \implies \mathsf{ciu\_jTy}\ \Sigma\ \Gamma\ t\ (\mathsf{beta\_reduce\_once}(t))\ A.$$

7.3 CaseCase: commuting nested case

The file `theories/Transform/CaseCase.v` implements a commuting conversion for nested cases.

Canonicalisation example (term-level). Consider two terms that differ only by administrative placement of case analysis:

$$t_1 \triangleq \mathsf{case}^J((\mathsf{case}^I(x; C; \overline{br_I})); D; \overline{br_J}) \qquad \text{and} \qquad t_2 \triangleq \mathsf{case}^I(x; C; \overline{br'_I}).$$

The CaseCase transformation deterministically rewrites t_1 to t_2 by distributing the outer case into each inner branch. Under our notion of cyclic-object equality (same extracted term up to typed CIU, Section 5), this means t_1 and t_2 share the same canonical representative after one driving step:

$$\mathsf{ciu\_jTy}(\Sigma, \Gamma, t_1, t_2, A).$$

This is a minimal but important instance of "bureaucracy elimination": two syntactic presentations of the same computation/induction structure are identified by a CIU-preserving commuting conversion.

Informally:

$$\mathsf{case}^J((\mathsf{case}^I(x; C; \overline{br_I})); D; \overline{br_J}) \quad \rightsquigarrow \quad \mathsf{case}^I(x; C; \overline{br'_I})$$

where each inner branch is transformed to:

$$br'_I \triangleq (\text{rebuild a lambda prefix for the constructor arity}) \ \ldots \text{ then } \mathsf{case}^J((br_I\ \overline{a}); D; \overline{br_J}).$$

In the development, the constructor arity/types are obtained from the inductive signature environment, and the branch transform is implemented by a function `commute_branch_typed_rec`.

7.4 Information propagation in case scrutinee

When the scrutinee of a case is known to be a constructor, we can propagate this information in two complementary ways:

Motive propagation. Instantiate the motive with the known constructor:

$$\mathsf{case}^I((\mathsf{roll}^I_c(\overline{a})); C; \overline{br}) \quad \rightsquigarrow \quad \mathsf{case}^I((\mathsf{roll}^I_c(\overline{a})); C[\mathsf{roll}^I_c(\overline{a})/x]; \overline{br})$$

This is semantically a no-op at runtime (the motive is erased by reduction), but it makes the instantiated motive explicit and can enable further simplifications. The transformation is implemented as `propagate_motive_once` in `theories/Transform/CaseCase.v`.

Scrutinee propagation into branches. More aggressively, we can specialize the selected branch knowing the scrutinee's constructor form. When the scrutinee is $\mathsf{roll}_c^I(\overline{a})$, the branch br_c will receive arguments $\overline{a}$ at runtime. We can transform br_c by:

1. Computing the branch's result type with the motive fully instantiated: $C[\mathsf{roll}_c^I(\ldots)/x]$
2. Reducing/simplifying this instantiated type (e.g. if C itself contains a case-of-constructor, it can reduce)
3. Adjusting the branch body to take advantage of the simplified result type

Formally:

$$\mathsf{case}^I((\mathsf{roll}_c^I(\overline{a})); C; \overline{br}) \quad \rightsquigarrow \quad \mathsf{case}^I((\mathsf{roll}_c^I(\overline{a})); C; [br_0, \ldots, br'_c, \ldots])$$

where br'_c is the specialized version of br_c.

Example. If the motive is $C = \lambda x.\ \mathsf{case}^I(x; D; \ldots)$, then knowing $x = \mathsf{roll}_c^I(\overline{a})$ allows us to reduce the inner case in the result type, potentially unlocking further simplifications in the branch body.

This transformation is CIU-preserving because the branch's observable behavior (which arguments it receives, what value it computes) is unchanged; we are only making the type information more concrete.

Implementation status: This transformation is implemented with a proved CIU preservation theorem in `theories/Transform/ScrutineeProp.v`.

7.5 Full normalization: driving under binders

For supercompilation-style transformations, we need to normalize terms fully, not just to weak-head normal form. This requires *driving under binders*: reducing subterms even when they appear under λ, Π, or case motives.

Driving strategy. The supercompiler (see `theories/Transform/Supercompile.v`) uses a fuel-limited driving function that applies the following transformations iteratively (cf. the supercompilation bundle above):

1. **Partial evaluation**: head reductions (beta, fix-unfolding, iota)
2. **Information propagation**: propagate known scrutinee into motive/branches
3. **Case distribution**: commute nested case expressions (CaseCase)
4. **Unfolding**: expand cyclic structure to expose more redexes
5. **Generalise and fold**: prevent divergence and re-introduce sharing/back-links
6. **Descend under binders**: recursively drive subterms under λ, Π, and case motives

Together these steps constitute the intended supercompilation-based normalization procedure for cyclic proofs.

Eta-expansion. A key question for full normalization is whether we need eta-rules:

$$\begin{array}{ll} \text{(eta-lambda)} & \lambda x.\ (t\ x) \equiv t \quad \text{if } x \notin FV(t) \\ \text{(eta-case)} & \mathsf{case}^I(t; C; \overline{br}) \equiv t \quad \text{if each } br_c = \mathsf{roll}^I_c(\ldots) \end{array}$$

Current status: The implementation does *not* require eta-rules for correctness. Eta-expansion can be added as an optional normalization step if needed for canonical forms, but observational equivalence (CIU) is preserved without it.

7.6 Unfolding and refolding of cycles

Beyond term-level rewrites, cyclic proof objects admit transformations that restructure graph sharing and cycles:

- **Unfold**: expand a back-link or a shared subgraph, temporarily increasing size but exposing computation.
- **Fold/refold**: identify repeated sub-derivations and introduce a back-link, or rearrange cycle boundaries (e.g. SCC-normal forms).

These transformations are the point where cyclicity pays off. Because recursion is represented structurally (by cycles) rather than by a fixed syntactic operator, refolding can change *which* cyclic structure represents a proof without changing what it proves.

Why refolding is sound. Refolding is only sound when the resulting cycles satisfy a global progress condition. In our development, cyclic objects are packaged together with explicit progress evidence (Section 6); transformations that introduce or move back-links are required to transport or rebuild this evidence. Operationally, this blocks unsound refoldings that would validate an ill-founded "loop": any directed cycle must contain a designated progress edge, and a well-founded ranking (or trace-style witness) must justify that the cycle makes genuine progress.

Thus, the normalization story factors cleanly: local CIU theorems justify term-level rewrites on extracted terms, while progress-preservation arguments justify the fold/refold steps on graphs.

8 Key theorems (mechanized)

This section states the main semantic preservation theorems mechanized in the Coq development. Our semantic equivalence is CIU-style observational equivalence at a typing judgement (values as observables).

8.1 Typed CIU equivalence

Write $\mathsf{ciu\_jTy}(\Sigma, \Gamma, t, u, A)$ for typed CIU equivalence (see `theories/Equiv/CIUJudgement.v`). Intuitively, t and u are equivalent at type A if, for every well-typed closing substitution by values, they terminate to the same values.

8.2 Head beta reduction

Let β_1 be head call-by-name beta reduction (`theories/Transform/BetaReduce.v`).

Theorem 8.1 (Head beta preserves typed CIU)**.** *For all* Σ, Γ, t, A*,*

$$\Sigma;\ \Gamma \vdash t : A \implies \mathsf{ciu\_jTy}(\Sigma, \Gamma, t, \beta_1(t), A).$$

This corresponds to the proved Coq theorem `ciu_jTy_beta_reduce_once`.

8.3 Read-off and extraction round-trip

Let read_off_raw compile a source term to a fix-free cyclic graph, and let extract_read_off be the specialized extraction back to terms. The Coq development proves a strong round-trip equation (`theories/Transform/ReadOffExtractCorrectness.v`).

Theorem 8.2 (Round-trip identity)**.** *For every term t,*

$$\mathsf{extract\_read\_off}(t) = t.$$

An immediate corollary is CIU preservation:

Theorem 8.3 (Round-trip preserves typed CIU)**.** *For all Σ, Γ, t, A,*

$$\mathsf{ciu\_jTy}(\Sigma, \Gamma, t, \mathsf{extract\_read\_off}(t), A).$$

These correspond to the proved Coq theorems `extract_read_off_id` and `extract_read_off_ciu`.

8.4 Case–case commuting conversion

Let $\mathsf{CaseCase}(\Sigma, t)$ be the one-step commuting conversion for nested case (`theories/Transform/CaseCase.v`).

Theorem 8.4 (CaseCase preserves typed CIU)**.** *For all Σ, Γ, t, A,*

$$\Sigma;\, \Gamma \vdash t : A \implies \mathsf{ciu\_jTy}(\Sigma, \Gamma, t, \mathsf{CaseCase}(\Sigma, t), A).$$

This corresponds to the proved Coq theorem `ciu_jTy_commute_case_case_once`.

8.5 Lifting transformations

As a first "lifting" result, the repository contains a term-level wrapper that uses the above CIU theorem to justify the CaseCase transform on extracted cyclic objects.

Theorem 8.5 (CaseCase transform preserves typed CIU (by reduction to term theorem))**.** *For all Σ, Γ, p, A, if p is well-typed at A, then applying CaseCase to p preserves typed CIU.*

This corresponds to the proved Coq theorem `case_case_transform_preserves_equiv`.

8.6 Ranking-based global progress (proved)

The cyclic proof layer uses a global condition to validate cycles. One convenient instantiation uses a natural-number ranking that is monotone on all edges and strictly decreasing on designated progress edges.

Proposition 8.6 (Natural-number ranking implies progress condition)**.** *If a finite proof graph admits a rank $\rho : V \to \mathbb{N}$ such that ranks never increase along edges and strictly decrease along progress edges, and every directed cycle contains a progress edge, then the global progress condition holds.*

This corresponds to `progress_ok_nat` in `theories/Progress/Measures.v`.

9 Canonical forms via supercompilation-style control

The long-term goal is a theory of *canonical cyclic proofs*: intensional representatives stable under semantics-preserving transformations. In this project, the intended normalization procedure is explicitly inspired by supercompilation [9, 7].

9.1 Control relation (homeomorphic embedding)

Supercompilation requires a termination control (a "whistle") preventing infinite unfolding. The repository implements a homeomorphic embedding relation on terms and extends it to typed judgements (`theories/Progress/Embedding.v`). We write $j_1 \preceq j_2$ for judgement embedding.

The classical meta-theorem (by Kruskal-style arguments [5]) is that homeomorphic embedding is a well-quasi-order on finite terms, which implies that an infinite sequence must contain an embedding pair. Operationally, this justifies the control rule:

> If a newly produced configuration embeds a previous one, stop unfolding and refold (generalize/backlink) instead.

9.2 Reductions "modulo unfolding"

Let $\to$ be the operational step relation on terms (Section 4). Let $\rightsquigarrow$ denote a single *unfolding* step at a cyclic back-link (or, in the source calculus, unfolding a `fix`). We treat $\rightsquigarrow$ as admissible but controlled: it is only permitted when it does not violate the control relation.

9.3 Canonical form (intended)

We say a cyclic object is in *canonical form* if:

(1) (No further reductions) There is no reduction step $p \to p'$ available in the cyclic object.

(2) (Unfolding is controlled) Any unfolding step $p \rightsquigarrow p_1$ that would enable further reductions $p_1 \to^+ p_2$ is forbidden by the control relation: along the would-be unfolding/reduction trace, some configuration embeds a previous one (a "whistle" blows), so the procedure must refold rather than continue unfolding.

This definition mirrors supercompilation practice: normal forms are not merely "stuck"; they are *maximally reduced subject to a termination control*. The presence of cycles (back-links) turns this into a canonicalization problem for graphs rather than trees: refolding chooses cycle targets and substitutions, and the global progress condition validates that the chosen cycles are productive/sound.

9.4 Connection to transformations

Under this view, transformations like head beta and CaseCase are the "driving" steps, while read-off/extract and fold/refold manipulate the sharing and cycles. The control relation (embedding on typed judgements) is the mechanism that prevents infinite unfolding and makes the canonical form finite.

10 Conclusion

This project is motivated by a proof-engineering problem that is easy to recognize in large dependently typed developments: *semantic computation is often blocked by bureaucratic syntax.* Even when two derivations represent the same underlying recursive reasoning, the boundaries of `fix` binders, the choice of induction schema, and the placement of administrative rewrites can force the proofs to remain syntactically distinct. Our claim is that cyclic proof objects provide a principled way to factor this bureaucracy out: recursion becomes *graph structure* (cycles plus explicit substitution evidence), while local steps remain ordinary CIC-style typing and computation.

Three concrete examples (drawn from the companion notes in `../cyclic-paper`) illustrate why this matters:

- **Cyclic even/odd reasoning as a proof graph.** A simple parity proof can be represented as a finite derivation with a back-link from a bud to its companion, carrying explicit substitution evidence (e.g. $\sigma : (n \mapsto m)$). The cyclic structure expresses the recursion directly, without committing to a particular `fix` term or induction schema.
- **Index bureaucracy from harmless program composition.** Composing traversals such as $\mathsf{map}\ g(\mathsf{map}\ f\ l)$ produces indices like $\mathsf{length}(\mathsf{map}\ g(\mathsf{map}\ f\ l))$ that are propositionally equal to $\mathsf{length}\ l$ but not definitionally equal, forcing explicit transports/casts. We view cyclic normalization (supercompilation-style driving plus fold/refold) as a way to normalise such index computations without strengthening kernel definitional equality.
- **Fix-free recursion via back-links with substitution witnesses.** In the extracted cyclic vertex language, there is no `fix` node label. Instead, recursive calls are explicit `nBack` nodes that point to (i) a cycle target and (ii) a linked-list substitution witness. This makes recursion refactorable: fold/refold can change cycle boundaries without changing local typing.

Against this background, the main contributions are:

- **A cyclic CIC calculus.** We give (to our knowledge) the first development of a CIC-style setting where proof objects are finite graphs with cycles, rather than trees with a fixed recursion/induction discipline.
- **Cyclic normalization as supercompilation.** We make a direct connection between cyclic proof normalization and supercompilation: driving steps correspond to local computation/commuting conversions and information propagation, while fold/refold and back-links correspond to memoization, generalization, and controlled reuse, governed by a termination control (homeomorphic embedding).
- **Verified meta-theory and transformation correctness.** Substantial parts of the pipeline and its supporting theorems are verified in Coq, including key semantic preservation results and the read-off/extraction round-trip theorem.

Verified results and current status. The mechanized development currently includes:

- a proved CIU preservation theorem for head beta reduction (Theorem 8.1);
- a proved CIU preservation theorem for the CaseCase commuting conversion (Theorem 8.4);
- a proved round-trip identity theorem for read-off and extraction (Theorem 8.2) and its CIU corollary (Theorem 8.3);

- a proved ranking-based sufficient condition for the global progress predicate (Proposition 8.6).

Future work is to complete the remaining transformation theorems (including more of the supercompilation bundle), strengthen the canonical-form theory, and refine the notion of canonical cyclic representatives (graph equivalence and refolding invariants).

References

[1] Bahareh Afshari and Dominik Wehr. Abstract cyclic proofs. *Mathematical Structures in Computer Science*, 34:552–577, 2024.

[2] Andrea Asperti, Wilmer Ricciotti, Claudio Sacerdoti Coen, and Enrico Tassi. A compact kernel for the calculus of inductive constructions. *Sādhanā*, 34(1):71–144, 2009.

[3] James Brotherston. Cyclic proofs for first-order logic with inductive definitions. In *TABLEAUX*, 2006.

[4] James Brotherston and Alex Simpson. Sequent calculi for induction and infinite descent. In *LICS*, 2011.

[5] Joseph B. Kruskal. Well-quasi-ordering, the tree theorem, and Vazsonyi's conjecture. *Transactions of the American Mathematical Society*, 1960.

[6] Michael Leuschel. Homeomorphic embedding for online termination of partial deduction. In *Logic-Based Program Synthesis and Transformation*, 2002.

[7] Morten Heine Sørensen and Robert Glück. An introduction to supercompilation. In *Partial Evaluation and Semantics-Based Program Manipulation*, 1995. Often cited introductory survey; venue details vary by edition.

[8] Christoph Sprenger and Mads Dam. On the structure of inductive reasoning: Circular and tree-shaped proofs in the μ-calculus. In *Foundations of Software Science and Computation Structures (FoSSaCS 2003)*, volume 2620 of *Lecture Notes in Computer Science*, pages 425–440. Springer, 2003.

[9] Valentin F. Turchin. *The Concept of a Supercompiler*. Springer, 1986. Classic reference introducing supercompilation.